

Case sheet: Agentic delivery pilot — multi-tenant SaaS

Version 2.1 · September 2026 · supersedes v1

In one sentence: I built an orchestration framework that lets AI agents deliver a real application stage by stage, with independent review and a human approving each gate — this sheet shows the measured result of its first pilot, including a pre-production readiness assessment of my own work.

Context

A vehicle-fleet damage-documentation product (multi-tenant SaaS: web + PWA, offline-capable). Earlier manual prototyping via AI chat tools stalled: 4 commits in 4 months, non-functional codebase.

Approach

The same brief was fed into my two-layer agentic orchestration framework: an interactive design phase (gates G0–G3 producing domain model, architecture, safety manifest, API contract and delivery roadmap) followed by a headless delivery loop — per-stage state machine, review by an AI model from a different vendor family than the author ("cross-family review"), deterministic sanity and scope checks, crash-safe resumable state. Human input is limited to gate approvals; an autonomy lock in the framework state enforces this.

Safety manifest = application-level guardrails (authorization, tenant data boundaries) — not functional safety in the ISO 26262 sense.

Delivery, measured from the repository

- 7 delivery stages: auth/tenancy → fleet → shifts and offline queue → damage reports, photos, notifications → history and dashboard → PDF/Excel export → PWA hardening
- **93 commits, 22 merged pull requests, 14 fix branches** (self-correction visible in history) in **~5 weeks**
- **19,075 lines of application code** (TypeScript/React, Prisma/PostgreSQL, NextAuth) plus **12,350 lines of tests**
- Human interventions: design-gate approvals and one approval per stage; no hand-written application code

Verification, measured after deployment

The pilot now runs in a production environment on its own domain.

- **385 automated tests across 46 files, plus 15 browser scenarios**
- **Two rounds of a scripted production walkthrough**, executed against the live environment and recorded in a protocol: full operational cycle (vehicle, driver assignment, shift with walkaround photography, damage report through its status transitions, exports, both audit journals), negative checks, mobile viewport, and re-authentication to prove persistence

- **Server-side authorization verified from outside the UI**, not assumed from a hidden menu item: an unauthenticated machine request returns `401` with a `problem+json` body, a browser navigation returns a redirect to sign-in, and an authenticated user without the required role receives `403 forbidden-role`
- **Four findings from round one were addressed; round two confirmed three of those fixes and recorded the fourth as still open** — a mobile layout defect that was only partially resolved. It is written up as failed, not as fixed

Two blockers in round one were evidence gaps rather than product defects, and both were closed with evidence: export files were retrieved and opened (rendered PDFs with embedded fonts, spreadsheets opened in Excel, hashes inventoried), and the authorization checks above were completed from the command line.

The protocol refuses to record a step as passed without evidence. An independent reviewer of the round-one protocol found one material inaccuracy — an unverified precondition that had not been reflected in a step's result — and the author downgraded his own result accordingly rather than defending it.

Pre-production readiness assessment

After the walkthrough, the same discipline was turned on the delivery itself: an audit of the repository and the running environment, producing findings ranked by what actually blocks go-live rather than by how easy they are to fix.

The assessment is deliberately uncomfortable reading. Its most consequential finding is not a bug:

Backups of the database and of stored evidence photographs were running, and verifying restorability was an explicit requirement in the customer's brief. **That verification had never been performed** — and the word "backup" did not appear anywhere in the repository. Backup being switched on is not the same thing as restore being proven.

Nothing in the test suite, the walkthrough or the deployment pipeline would have caught this, because every one of them looks at what the system does rather than at what would happen when it fails. It surfaced only because the readiness assessment asked a different question: not "does it work", but "which of the customer's requirements has no evidence behind it".

Alongside it the assessment records operational hardening items — platform credential design, continuous integration, runtime monitoring, and end-user documentation as a contractual deliverable — each with an owner and a position relative to go-live. One of them is tied to a concrete consequence: without continuous integration the test suite ran only on one machine, and a mobile defect reached production before the walkthrough found it.

Specific control-level findings on a running system are not published here; the full assessment is available for technical review under NDA.

Why it matters

This is governed delivery, not "AI wrote a script": design authority separated from implementation, independent cross-vendor review, auditable state, and verification that reports its own gaps. The same control pattern applies to infrastructure as code, migrations, documentation and compliance-checked automation.

The readiness assessment is the part worth borrowing. A delivery is not finished when the features work. It is not finished until someone has asked, in writing, which requirements are still carried by assumption instead of evidence — and has published the answer.

Repository and full run artifacts (stage prompts, scope checks, review verdicts, walkthrough protocols) available for technical review under NDA.